

GITHUB COPILOT

MCP

Protocolo de
contexto de modelo

Servidor MCP de GitHub

Integración hospedada, segura y escalable de GitHub en
flujos de trabajo asistidos por IA

Configuración en Visual Studio Code · Uso con Copilot Chat · Modo agente

Agenda

01 Qué es MCP y el servidor MCP de GitHub

03 Por qué usar el servidor hospedado

05 OAuth, PAT y Docker

07 Uso con Copilot Chat y modo agente

02 Cómo se conectan los clientes MCP

04 Configuración en Visual Studio Code

06 Solución de problemas

08 Procedimientos recomendados

SECCIÓN 1

MCP y el servidor MCP de GitHub

Fundamentos del Protocolo de contexto de modelo y su integración con GitHub

El servidor MCP de GitHub

- Forma hospedada, segura y escalable de integrar GitHub en flujos de trabajo asistidos por IA.
- Basado en el Protocolo de contexto de modelo (MCP), introducido por Anthropic.
- Amplía GitHub Copilot y otras herramientas de IA para automatizar tareas y administrar repositorios.
- Mejora la experiencia de desarrollo con asistencia de IA y reconocimiento del contexto.
- Disponible para Visual Studio Code, con ampliación prevista a otros editores y plataformas.

Objetivos de aprendizaje

- Comprender qué son MCP y el servidor MCP de GitHub, y cuál es su utilidad.
- Configurar el servidor MCP de GitHub en Visual Studio Code.
- Usar el servidor MCP de GitHub con Copilot Chat para automatizar tareas de desarrollo.
- Identificar y resolver problemas comunes.

Prerrequisitos

Requeridos

- Cuenta de GitHub.
- Visual Studio Code u otro editor compatible con MCP.
- En organizaciones o empresas con Copilot Business o Copilot Enterprise, la directiva «Servidores MCP en Copilot» debe estar habilitada.

Opcionales

- Token de acceso personal (PAT) de GitHub para configuración avanzada y control de permisos.
- Docker para experimentar con una configuración de servidor local.

¿Qué es MCP?

Reduce la fricción de configuración y habilita funcionalidades como la evaluación de prioridades de problemas y la búsqueda semántica.

- Funciona como un estándar de conexión para herramientas de IA.
- Proporciona una forma coherente y segura de conectar modelos de IA con las herramientas y fuentes de datos que necesitan.
- **Acceso a herramientas:** biblioteca creciente de herramientas que los modelos de IA pueden utilizar.
- **Flexibilidad:** trabajar con diferentes proveedores de IA manteniendo flujos de trabajo coherentes.
- **Integración:** encaja con el entorno de desarrollo y los procesos existentes.

Cómo se conectan los clientes MCP

Un cliente MCP —Claude, un IDE u otra herramienta— puede interactuar con servidores y servicios de tres formas.

1

Comunicación local con datos locales

El cliente se comunica con un servidor MCP en la propia máquina, conectado a archivos o bases de datos locales.

Cuándo usarlo: desarrollo local o datos privados.

2

Servidor local como puente a servicios remotos

El cliente se conecta a un servidor local que se comunica con un servicio remoto mediante sus API web.

Cuándo usarlo: cuando conviene caché, comprobaciones de seguridad o preprocesamiento.

3

Comunicación remota a través de Internet

El cliente se conecta a un servidor MCP completamente remoto, que se comunica con otros servicios mediante API web.

Cuándo usarlo: procesos en la nube, SaaS o integraciones de terceros solo en línea.

SECCIÓN 2

¿Por qué usar el servidor MCP de GitHub?

Del servidor local autogestionado al servidor hospedado por GitHub

Servidor local frente a servidor hospedado

Servidor MCP local

- Requiere normalmente Docker.
- Administración manual de tokens.
- Configuración manual que ralentiza la puesta en marcha.
- Dificulta la integración con clientes web como GitHub.com.

Servidor hospedado de GitHub

- Conexión rápida y sencilla, sin archivos Docker ni configuración manual.
- Herramientas de IA como Copilot Chat en web y móvil.
- Escala los proyectos a medida que crecen.
- Inicio de sesión empresarial seguro, búsqueda semántica de código y correcciones automatizadas.

Ventajas del servidor hospedado

- Elimina la necesidad de archivos de configuración manual o Docker.
- Proporciona autenticación OAuth sencilla con un solo clic.
- Permite trabajar en entornos web, de escritorio y móviles.
- Admite proveedores de identidad empresariales como Entra y Auth0.
- Escala automáticamente según las necesidades de uso.

El servidor MCP en acción

+30

herramientas disponibles
actualmente en el servidor MCP
de GitHub

- Analizar y resumir código para comprender proyectos.
- Crear y administrar problemas y solicitudes de incorporación de cambios.
- Automatizar la evaluación de prioridades del repositorio y el seguimiento de tareas.
- Agregar problemas, editar archivos y crear ramas.
- Clasificar pull requests y problemas para priorizarlos.
- Servidor de código abierto que conecta Copilot y otras herramientas de IA directamente con los repositorios.



SECCIÓN 3

Configuración en Visual Studio Code

OAuth, token de acceso personal, Docker y solución de problemas

Vías de configuración

El servidor MCP de GitHub se integra en el entorno de desarrollo mediante cualquiera de estas configuraciones.

1

OAuth

Autorización con GitHub en un solo clic desde la paleta de comandos de VS Code. Es la vía recomendada y más rápida.

2

Token de acceso personal

Configuración avanzada con control de permisos mediante un PAT con ámbito de repositorio y lectura de paquetes.

3

Local con Docker

Uso local del servidor mediante la imagen oficial. Requiere Docker en ejecución y autenticación por PAT; OAuth no se admite.

Configuración mediante OAuth

Tras la instalación, el servidor queda disponible en los proyectos de VS Code para automatizar tareas, administrar problemas y analizar código.

- 1 Abrir la paleta de comandos con Ctrl+Mayús+P (Windows/Linux) o Cmd+Mayús+P (Mac).
- 2 Escribir «MCP: agregar el servidor» y presionar Enter.
- 3 Seleccionar «HTTP (HTTP o Server-Sent Events)».
- 4 Introducir como dirección del servidor: <https://api.githubcopilot.com/mcp/>
- 5 Introducir un identificador de servidor o utilizar el valor predeterminado.
- 6 Elegir dónde guardar la configuración: de usuario o del área de trabajo.
- 7 Autorizar con GitHub mediante OAuth: seleccionar «Permitir» e iniciar sesión si se solicita.

Configuración mediante token de acceso personal

- 1 Crear un PAT con el ámbito de repositorio y lectura de paquetes en la cuenta de GitHub.
- 2 Seguir los pasos anteriores, pero cancelar OAuth cuando se solicite.
- 3 Agregar en el archivo de configuración el encabezado de autorización con el token de entrada.
- 4 Agregar un mensaje de entrada de tipo promptString con password activado para solicitar el token de forma segura.
- 5 Reiniciar el servidor MCP en VS Code e introducir el PAT cuando se solicite.
- 6 El servidor MCP utilizará el PAT para la autorización.

Fragmentos de configuración del PAT

- **Encabezado de autorización:** "headers": { "Authorization": "Bearer \${input:github_token}" }
- **Mensaje de entrada:** "inputs": [{ "id": "github_token", "type": "promptString", "description": "GitHub Personal Access Token", "password": true }]
- El indicador password: true evita que el token quede visible o almacenado en texto plano.
- Tras guardar los cambios, reiniciar el servidor MCP para que la nueva configuración se aplique.

Configuración local con Docker

Para uso local el servidor requiere Docker y autenticación mediante PAT; OAuth no se admite en esta configuración.

- 1 Confirmar que Docker está instalado y ejecutándose.
- 2 Generar un PAT con los ámbitos necesarios.
- 3 Definir la entrada `github_token` de tipo `promptString` con `password: true`.
- 4 Configurar el servidor github con el comando `docker` y la imagen `ghcr.io/github/github-mcp-server`.
- 5 Asociar la variable `GITHUB_PERSONAL_ACCESS_TOKEN` al token introducido.
- 6 Reiniciar el servidor MCP e introducir el PAT cuando se solicite.

GitHub Enterprise Server y restricciones de PAT

- En GitHub Enterprise Server con restricciones de PAT, solo se puede acceder a los ámbitos de API permitidos por la directiva de la organización.
- Si todos los puntos de conexión están restringidos, el servidor MCP no estará disponible.
- Conviene revisar con antelación la directiva de la organización antes de planificar la configuración local.
- En organizaciones con Copilot Business o Copilot Enterprise, la directiva «Servidores MCP en Copilot» debe estar habilitada.

Solución de problemas

- Confirmar que se inició sesión en GitHub desde Visual Studio Code.
- Si se utiliza un PAT, verificar que tenga los ámbitos correctos y esté escrito correctamente.
- Revisar la configuración para detectar errores tipográficos o campos faltantes.
- Si se utiliza Docker, comprobar que esté instalado y ejecutándose.
- Reiniciar VS Code o el servidor MCP para resolver problemas temporales de conexión.



SECCIÓN 4

Uso con Copilot Chat y modo agente

De responder a una petición única al ciclo de explorar, adaptar y refinar

Uso con Copilot Chat

Los servidores MCP amplían Copilot conectándolo con herramientas y recursos externos; con el modo agente puede planear, ejecutar y refinar flujos de trabajo.

- 1 Abrir el Chat de Copilot en Visual Studio Code y cambiar al modo agente para activar las herramientas del servidor MCP.
- 2 Seleccionar «Seleccionar herramientas» para visualizar las funcionalidades disponibles.
- 3 Utilizar lenguaje natural: crear un nuevo problema, resumir un repositorio u obtener información sobre el trabajo.
- 4 Seguir las indicaciones de Copilot Chat para completar las tareas.

Modo agente y MCP

Funcionalidades agénticas

- Trabajar de forma independiente mediante flujos de varios pasos sin instrucciones constantes.
- Tomar decisiones y elegir herramientas o enfoques según el contexto.
- Adaptarse, mejorar e iterar en función de los resultados y comentarios.
- Copilot actúa como un colaborador que comprende el contexto general.

Cómo MCP amplía el modo agente

- Acceder directamente a datos externos, API y herramientas empresariales.
- Mantener el contexto entre plataformas sin cambiar de aplicación.
- Completar bucles agentes: buscar información, analizar resultados y ejecutar los siguientes pasos de forma dinámica.

Ventajas de combinar MCP con el modo agente

El flujo pasa de responder a una única petición a trabajar mediante un ciclo de explorar, adaptar y refinar.

1

Contexto extendido

Copilot puede obtener información de varios sistemas además del editor de código.

2

Menor esfuerzo manual

Automatiza tareas rutinarias como abrir problemas, administrar flujos de trabajo o ejecutar comprobaciones.

3

Integración sin problemas

Permite realizar tareas que abarcan herramientas y plataformas sin conectores personalizados ni cambios constantes de aplicación.

Procedimientos recomendados

- **Definir claramente los objetivos:** especificar qué debe lograr Copilot y cuál debe ser el resultado final.
- **Proporcionar contexto:** incluir información sobre el proyecto o flujo de trabajo, como vínculos, referencias o pasos anteriores.
- **Establecer límites:** indicar si debe detenerse en la planificación sin realizar cambios y limitar las herramientas MCP activas.
- **Pedir confirmación:** antes de grandes cambios, solicitar que Copilot resuma el plan para aprobarlo o refinarlo.
- **Usar archivos de solicitud o instrucciones:** orientar el comportamiento de Copilot con servidores MCP específicos y mantener coherencia.

Puntos clave

MCP es un estándar de conexión seguro y coherente entre modelos de IA, herramientas y fuentes de datos.

El servidor hospedado de GitHub elimina Docker y la configuración manual, con OAuth de un solo clic.

La configuración en VS Code admite OAuth, token de acceso personal y, opcionalmente, Docker en local.

Con el modo agente, Copilot planea, ejecuta y refina flujos de trabajo de varios pasos.

Objetivos claros, contexto, límites y confirmación son la base de un uso eficaz.